

DEPLOY AND OPERATE ODOO IN PRODUCTION

Infrastructure & DevOps

Docker, VPS, Nginx, SSL, and workers

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: Docker Fundamentals — 11 topics
- 2 Section 2: VPS and Reverse Proxy — 9 topics
- 3 Section 3: Odoo Performance Settings — 12 topics



1 Section 1

Docker Fundamentals

1.1 Containers vs VMs

- A container packages an app process and its dependencies.

Container mental model

```
app process + image filesystem + mounted volumes
```

- A VM packages a full guest operating system.

VM mental model

```
guest OS + virtual disk + virtual network + app
```

1.2 Install Docker

- Install Docker from trusted package sources.

Version checks

```
docker --version  
docker compose version
```

- Verify both the engine and compose plugin.

Service check

```
sudo systemctl enable --now docker  
sudo systemctl status docker
```

1.3 docker run, ps, and logs

- docker run starts a container from an image.

Run a test container

```
docker run --rm hello-world
```

- docker ps shows running containers.

Inspect containers

```
docker ps  
docker logs odoo-web
```

1.4 Images vs Containers

- An image is the reusable blueprint.

List images

```
docker images
```

- A container is a running or stopped instance.

List all containers

```
docker ps -a
```

1.5 Dockerfile Odoo Idea

- A Dockerfile documents how to build a custom Odoo image.

Minimal base

```
FROM odoo:18
USER root
RUN pip3 install --break-system-packages openpyxl
# ...
```

- Install Python and system dependencies in the image.

Copy addons

```
COPY ./custom_addons /mnt/extra-addons
```

1.6 Volumes

- Volumes keep data outside disposable containers.

Named volume

```
volumes:  
  odoo-data:  
  pg-data:
```

- Odoo needs persistence for filestore and PostgreSQL data.

Mount volume

```
services:  
  web:  
    volumes:  
# ...
```

1.7 Docker Network

- Docker networks let containers reach each other by service name.

Compose network

```
networks:  
  odoo-net:
```

- Keep databases off the public internet.

Database host name

```
db_host = db  
db_port = 5432
```

1.8 Compose File

- Compose describes multiple services in one YAML file.

Compose command

```
docker compose up -d  
docker compose ps
```

- It is useful for Odoo, PostgreSQL, and helper services.

Service skeleton

```
services:  
  web:  
    image: odoo:18  
# ...
```

1.9 Odoo and PostgreSQL Compose

- Odoo needs a PostgreSQL database service.

Odoo service

```
web:
  image: odoo:18
  depends_on:
# ...
```

- Use environment variables for database connection settings.

PostgreSQL service

```
db:
  image: postgres:16
  environment:
# ...
```

1.10 .env File

- .env keeps environment-specific values out of compose YAML.

Example .env

```
ODOO_PORT=8069
POSTGRES_USER=odoo
POSTGRES_PASSWORD=change_me
```

- Do not commit real production secrets.

Compose reference

```
ports:
  - "${ODOO_PORT}:8069"
environment:
# ...
```

1.11 Upgrade Containers Safely

- Back up the database and filestore before upgrading.

Pull and restart

```
docker compose pull
docker compose up -d
```

- Pull new images in staging first.

Backup first

```
pg_dump -Fc odoo_prod > odoo_prod.dump
tar -czf filestore.tgz /var/lib/odoo/filestore
```

2 Section 2

VPS and Reverse Proxy

2.1 Choose VPS Specs

- Size the VPS for users, workers, database, and backups.

Small training host

2 vCPU
4 GB RAM
60 GB SSD

- Prefer SSD storage for PostgreSQL.

Busier host

4 to 8 vCPU
16 GB RAM
separate backup storage

2.2 SSH Hardening

- Use SSH keys instead of passwords.

sshd settings

```
PasswordAuthentication no  
PermitRootLogin no
```

- Disable direct root login when possible.

Reload SSH

```
sudo sshd -t  
sudo systemctl reload ssh
```

2.3 /opt/odoo Layout

- A predictable directory layout helps operations.

Common layout

```
/opt/odoo/  
  config/  
  custom_addons/  
# ...
```

- Separate config, addons, backups, and logs.

Ownership idea

```
sudo chown -R odoo:odoo /opt/odoo
```

2.4 Nginx Reverse Proxy

- Nginx receives public HTTP traffic.

Server block

```
server {  
    server_name odoo.example.com;  
    location / {  
# ...
```

- It forwards requests to Odoo on the private port.

Test config

```
sudo nginx -t  
sudo systemctl reload nginx
```

2.5 Proxy to 8069

- Port 8069 is the normal Odoo HTTP port.

Nginx proxy

```
location / {  
    proxy_set_header Host $host;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    # ...
```

- Expose it only to Nginx on production hosts.

Odoo config

```
proxy_mode = True  
xmlrpc_port = 8069
```

2.6 Longpolling 8072

- Odoo uses a separate longpolling or websocket endpoint.

Nginx longpolling

```
location /longpolling {  
    proxy_pass http://127.0.0.1:8072;  
}
```

- Route the longpolling path to port 8072 when configured.

Odoo port

```
longpolling_port = 8072
```

2.7 Certbot SSL

- Certbot can issue Let's Encrypt certificates.

Issue certificate

```
sudo certbot --nginx -d odoo.example.com
```

- Use HTTPS for every production Odoo site.

Renewal check

```
sudo certbot renew --dry-run
```

2.8 systemd odoo.service

- systemd keeps an Odoo process running on a VPS.

Service command

```
[Service]
User=odoo
ExecStart=/opt/odoo/venv/bin/python /opt/odoo/odoo-bin -c /etc/odoo.conf
# ...
```

- It defines the user, command, and restart behavior.

Operate service

```
sudo systemctl enable --now odoo
journalctl -u odoo -f
```

2.9 Cron Backup

- Backups should run automatically.

Cron line

```
0 2 * * * /opt/odoo/scripts/backup.sh
```

- Include both database dump and filestore.

Backup script idea

```
pg_dump -Fc odoo_prod > /backup/odoo_prod.dump  
tar -czf /backup/filestore.tgz /var/lib/odoo/filestore
```

3 Section 3

Odoo Performance Settings

3.1 Workers Idea

- Workers let Odoo handle multiple requests in parallel.

Worker setting

```
workers = 5  
max_cron_threads = 1
```

- Production deployments usually use workers.

No workers

```
workers = 0  
# common in simple development setups
```

3.2 Calculate Workers

- A common starting formula is CPU cores * 2 + 1.

Formula

```
workers = (cpu_cores * 2) + 1
```

- Adjust for memory, long requests, and cron load.

Example

```
2 CPU cores -> 5 workers
```

```
4 CPU cores -> 9 workers
```

3.3 Memory Limits

- Memory limits restart workers that grow too large.

Odoo limits

```
limit_memory_soft = 2147483648  
limit_memory_hard = 2684354560
```

- Set limits based on available RAM and worker count.

Container limit

```
deploy:  
  resources:  
    limits:  
# ...
```

3.4 Time Limits

- Time limits stop requests that run too long.

Config values

```
limit_time_cpu = 60  
limit_time_real = 120
```

- They protect workers from being stuck forever.

Import window idea

```
# raise limits only during a controlled import  
limit_time_real = 600
```

3.5 max_connections

- PostgreSQL max_connections must cover Odoo workers and tools.

PostgreSQL setting

```
max_connections = 200
```

- Too many connections can waste memory.

Odoo db setting

```
db_maxconn = 64
```

3.6 Missing Indexes

- Missing indexes make common searches slow.

SQL index

```
CREATE INDEX training_course_state_idx  
ON training_course (state);
```

- Look for slow queries before adding indexes.

Odoo field index

```
state = fields.Selection(  
    selection=STATES,  
    index=True,  
    # ...
```

3.7 pg_stat_statements

- pg_stat_statements records query statistics.

Enable extension

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

- It helps find slow and frequent SQL.

Find slow queries

```
SELECT calls, mean_exec_time, query  
FROM pg_stat_statements  
ORDER BY mean_exec_time DESC  
# ...
```

3.8 Redis Sessions Idea

- Redis can store shared session or cache data in some designs.

Redis service

```
redis:  
  image: redis:7  
  restart: unless-stopped
```

- It is useful when multiple Odoo instances need shared state.

Session concept

```
browser session -> load balancer -> Odoo nodes -> shared Redis
```

3.9 Load Balancing Idea

- Load balancing spreads traffic across Odoo nodes.

Upstream block

```
upstream odoo_backend {  
    server 10.0.0.11:8069;  
    server 10.0.0.12:8069;  
# ...
```

- All nodes must share database and filestore strategy.

Proxy to upstream

```
location / {  
    proxy_pass http://odoo_backend;  
}
```

3.10 Monitoring Overview

- Monitoring shows whether the system is healthy.

Useful signals

CPU, RAM, disk, HTTP errors, slow queries, backup age

- Track application, database, disk, and host metrics.

Basic command checks

```
df -h  
free -m  
systemctl status odoo
```

3.11 Logging Overview

- Logs explain what happened before and during failures.

Odoo log config

```
logfile = /var/log/odoo/odoo.log  
log_level = info
```

- Keep Odoo, Nginx, and PostgreSQL logs accessible.

Journal logs

```
journalctl -u odoo --since "1 hour ago"
```

3.12 Restore Testing

- A backup is only useful if it can be restored.

Restore database

```
createdb odoo_restore  
pg_restore -d odoo_restore odoo_prod.dump
```

- Restore tests prove database and filestore backups match.

Restore filestore

```
mkdir -p /var/lib/odoo/filestore/odoo_restore  
tar -xzf filestore.tgz -C /var/lib/odoo/filestore/odoo_restore
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory
07 · Infrastructure

